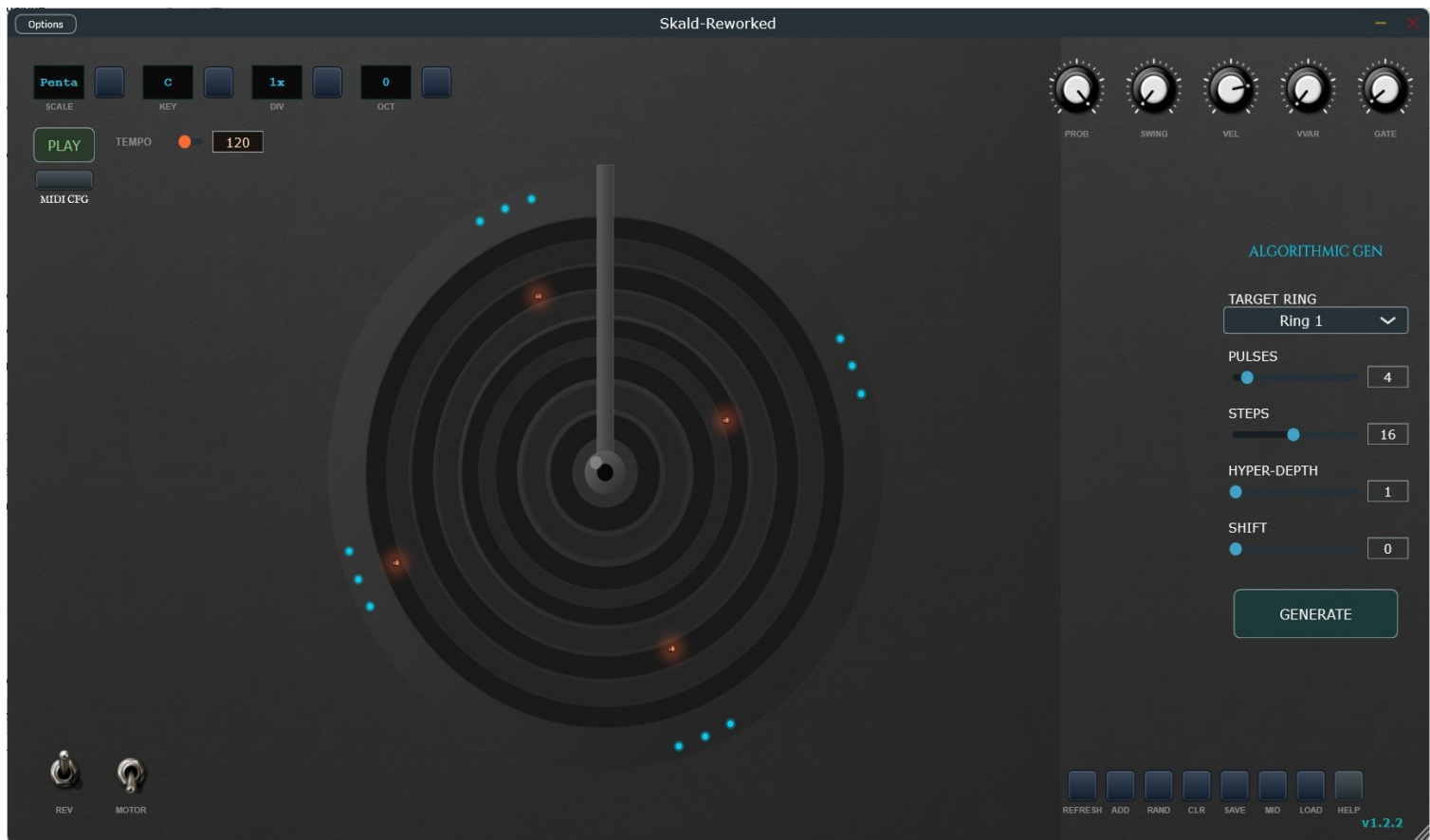


SKALD: User Manual & Technical Guide



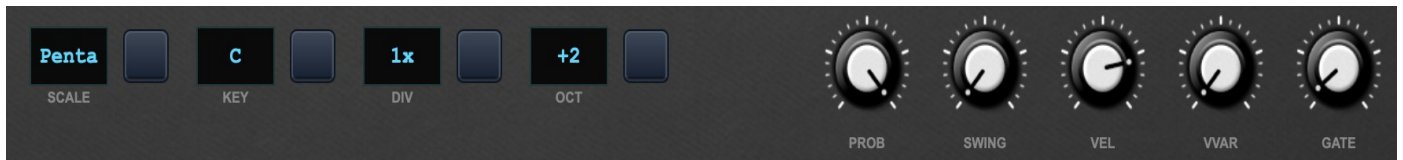
1. Introduction & Core Concept

Skald is a generative "Turntable" MIDI sequencer inspired by the organic movement of physical objects and the mathematical beauty of Euclidean rhythms.

Unlike linear step-sequencers, Skald operates on a circular logic. Imagine a vinyl record where notes are "dots" placed on concentric tracks. As the record rotates, these dots pass under a stationary "sensor arm" at the top (12 o'clock position), triggering MIDI notes.

The goal of Skald is to provide a performance-oriented environment where rhythm is discovered through geometry, probability, and rotation, bridging the gap between mechanical precision and human-like variation.

2. Interface Elements & Functions



Top Control Bar

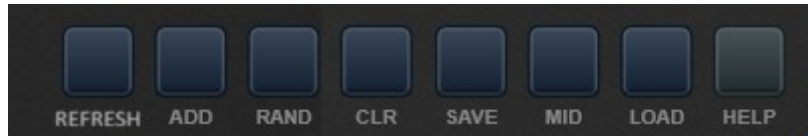
- **LED Displays (Scale, Key, Speed, Octave):** These high-visibility displays show your current harmonic and temporal settings. Click the **TAP** buttons next to them to cycle through scales or change quantization divisions.
- **Performance Knobs:**
 - **Probability:** Determines the chance (0-100%) that a note will actually trigger when passing the arm. Great for adding "breathing" to a pattern.
 - **Swing:** Introduces micro-timing offsets to "off-beat" notes for a human feel.
 - **Velocity & Variation:** Sets the base hit strength and adds a random range to prevent robotic repetition.
 - **Gate Time:** Controls the duration of the MIDI note-on message (from short staccatos to long drones).

The Turntable (Main Area)



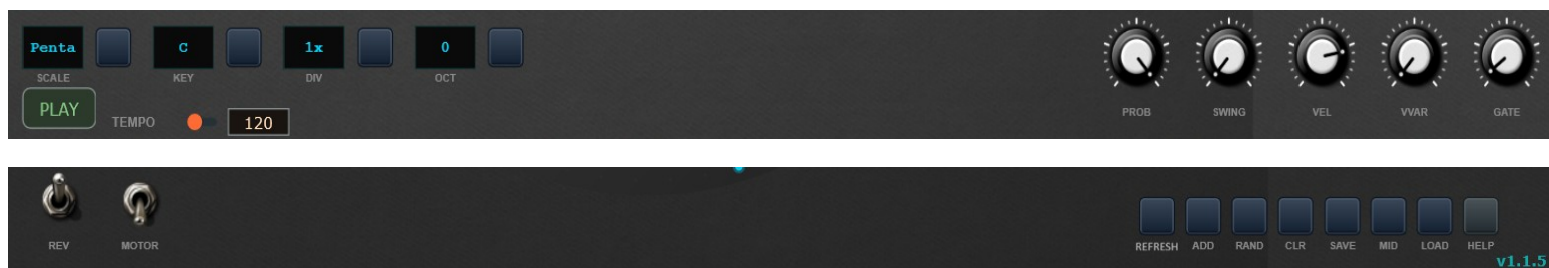
- **The Rings:** There are 10 zones (1 Scratch Ring, 8 Note Rings, 1 Center).
- **Visual Feedback:** Dots glow when passing the arm. A "Tracer" effect shows the active Gate Time, fading out as the note-off signal is sent.

Bottom Action Buttons



- **REFRESH:** Scans your system for MIDI output ports (Essential in Standalone mode).
- **ADD:** Enters "Add Mode" to place new dots on the rings.
- **RAND:** Generates a random scattering of dots for instant inspiration.
- **CLR (CLEAR):** Wipes the current pattern.
- **SAVE/LOAD:** Manages your pattern presets (.ttp files).
SAVE to save the pattern configuration, including the dots, and **LOAD** to load it into the app. You can choose any directory. Presets have a .ttp extension.
- **MID (Export):** Clicking this button allows you to export the current circular pattern as a standard MIDI file (.mid). A file browser will open, allowing you to **save the file to any location on your hard drive**. Once saved, you can manually import or drag this file into your DAW's timeline to further edit the performance or use it with other virtual instruments.
- **HELP:** this open a pdf manual (place the user guide in the same folder as standalone or plugin and not change his name (Skald_Manual.pdf)).

The higher and the lower areas of the app in the standalone version 1.1.5 seen in its full horizontal extension looks like this:



Algorithmic Gen Sidebar (Hyper-Euclidean)

The Algorithmic Gen section allows you to create complex patterns based on Euclidean geometry. This section allows you to generate mathematical patterns (Euclidean or based on density algorithms) in just a few seconds.

ALGORITHMIC GEN

TARGET RING

Ring 1

PULSES

4

STEPS

16

HYPER-DEPTH

1

SHIFT

0

GENERATE

1. Ring Selector:

Choose which ring (note) to operate on.
Select which ring to apply the generation to.

2. Pulses (*):

Set as many notes or strokes as you want.

3. Steps (*):

Sets the total number of steps over which to distribute the notes.

(*) Pulses & Steps: They define how many strokes to distribute evenly in a number of subdivisions (e.g. 3 strokes over 8 steps). Generates mathematically perfect Euclidean rhythms on the selected ring

4. Hyper-Depth:

Distributes notes across adjacent rings for melodic complexity. Adjusts pattern complexity. A special Skald feature that layers patterns for greater rhythmic complexity.

5. Shift:

Adjusts the rotational offset of the pattern.
Rotates the generated pattern to move the accents relative to the start of the cycle.

6. Button "Generate":

Applies the Hypereuclidean algorithm to the turntable to generate hypereuclidean patterns. Applica istantaneamente il pattern calcolato all'anello selezionato.



In Norse folk belief and culture, skalds (Old Norse: skald) were court poets active between the 9th and 11th centuries, figures of the highest prestige who composed and recited verses for kings, chieftains, and nobles. They were considered the guardians of the historical, cultural, and spiritual memory of the Scandinavian peoples.

3.1. Note Properties & MIDI Assignment

Skald allows deep per-note customization using **Modifier Keys** while dragging a dot (*remember to click on the dot before pressing the button corresponding to the desired controller*):

- **Placement:** Click **ADD**, then click any ring to place a dot. Drag it to change its timing (angle).
- **MIDI Channel (ALT + Drag):** Vertical movement changes the MIDI channel (1-16). The channel number is displayed directly on the dot. This allows one instance of Skald to trigger multiple instruments simultaneously.



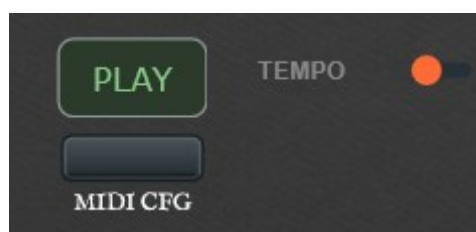
- **Velocity (SHIFT + Drag):** Adjusts the individual volume/intensity of that specific dot.
- **Gate Time (CTRL/CMD + Drag):** Adjusts how long that specific note stays "pressed."
- **Selection:** Click a dot to select it; the HUD (Heads-Up Display) in the top right will show precise values for its parameters.

3.2. MIDI LEARN Configuration Guide

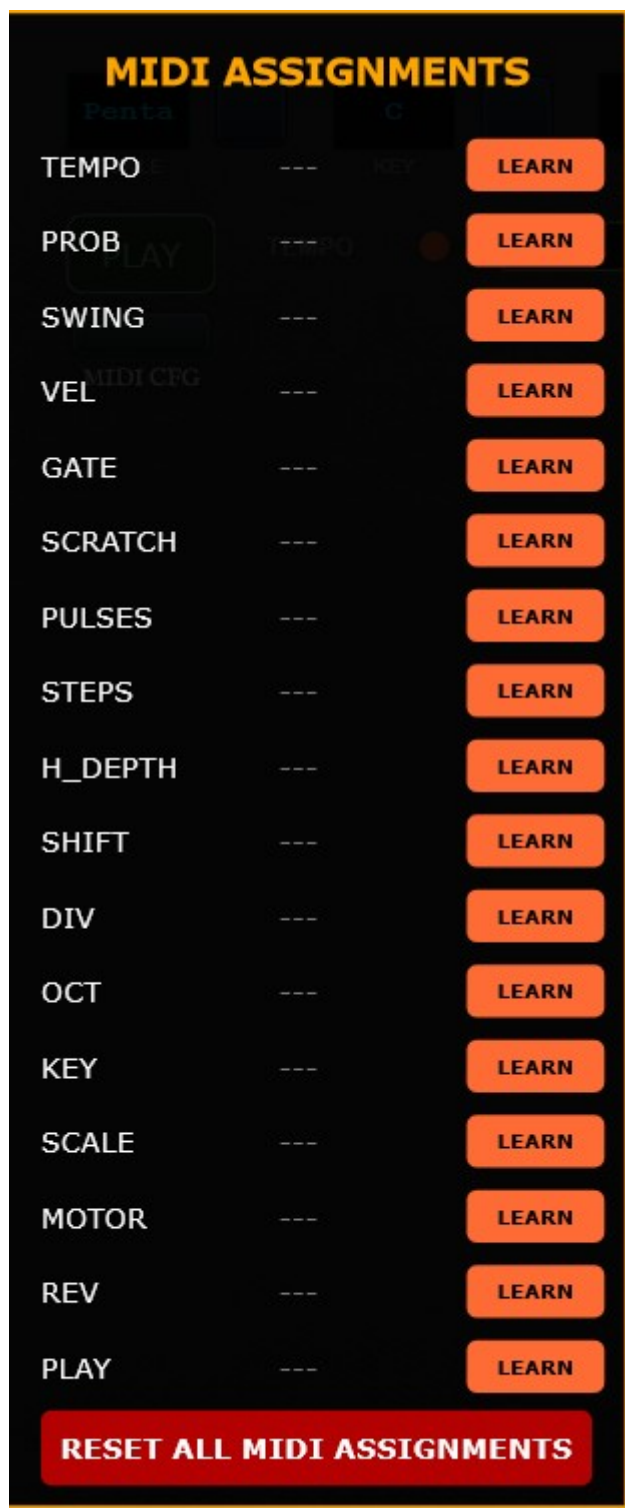
(the MIDI Learn function is only available from version 1.2.1)

1. Accessing the MIDI Matrix

Click the "**MIDI CFG**" button on the main interface to open the assignment popup. This window lists all mappable parameters, from core transport controls to algorithmic generation settings.



HARDWARE REQUIREMENTS RECOMMENDED FOR MIDI LEARN:



- **Tempo, Probability, Swing, Velocity, Gate, Scratch, Pulses, Steps, Depth, Shift**, are Continuous Controls parameters, these need Knobs & Faders. These respond to standard CC values (0-127). The plugin smoothly maps this range to the parameter's internal limits.

- **Motor, Reverse, Play** are toggle & binary control parameters. These need momentary buttons:
 - A CC value of **64 or higher** triggers the "ON" state.
 - A CC value **below 64** triggers the "OFF" state.

Note: If you use a physical toggle button that sends 127 on press and 0 on release, the plugin will react accordingly.

- **Division, Octave, Key** and **Scale**, are selectors (Selection & Stepped Controls). These parameters represent discrete choices (e.g., selecting between 12 keys or various scales). The MIDI range (0-127) is divided into equal "zones", (example: for the **Scale** selector, moving a knob will cycle through the available scales as the CC value increases).

- **Speed:** this parameter allows you to manipulate the turntable's rotation speed in real time, regardless of the overall BPM; it's an essential control for achieving time-stretching effects or manual pitch-shifting on the fly. The control is linearly mapped from 0.1x (minimum speed corresponding to a CC value of 0) to 4.0x (maximum speed corresponding to a CC value of 127). The normal playback speed of 1.0x equates to a CC value of approximately 30. For the best experience, we recommend using a high-resolution physical fader (which allows for classic pitch-riding techniques similar to those used on professional DJ turntables) or a large rotary encoder.

2. Mapping a Parameter (MIDI Learn)

To link a physical knob, fader, or button on your MIDI controller to a software parameter:

1. Locate the parameter in the list (e.g., **TEMPO** or **PROB**).
2. Click the "**LEARN**" button next to it. The button will turn red and display "**LISTENING...**".
3. Move the desired control on your MIDI hardware.
4. The system will instantly capture the CC (Continuous Controller) number, and the display will update to show "**CC XX**".

3. Deleting Assignments

- **Individual Reset:** To unbind a single parameter, **Right-Click** its orange "LEARN" (or "CC XX") button. The assignment will clear immediately, returning to "---".
- **Global Reset:** To wipe all MIDI mappings and start from scratch, click the large red "**RESET ALL MIDI ASSIGNMENTS**" button at the bottom of the popup.



Control Behaviors & Compatibility

The Skald engine intelligently handles different types of MIDI data based on the target parameter:

A. Continuous Controls (Knobs & Faders)

- **Parameters:** *Tempo, Probability, Swing, Velocity, Gate, Scratch, Pulses, Steps, Depth, Shift.*
- **Behavior:** These respond to standard CC values (0-127). The plugin smoothly maps this range to the parameter's internal limits.
- **Tip:** If your controller supports "high-resolution" or "stepped" knobs, Skald will accurately track the values, though visual updates on the UI may be slightly smoothed for clarity.

B. Toggle & Binary Controls (Buttons)

- **Parameters:** *Motor, Reverse, Play.*
- **Behavior:** Designed for momentary or toggle buttons.
 - A CC value of **64 or higher** triggers the "ON" state.
 - A CC value **below 64** triggers the "OFF" state.
- *Note:* If you use a physical toggle button that sends 127 on press and 0 on release, the plugin will react accordingly.

C. Selection & Stepped Controls (Selectors)

- **Parameters:** *Division, Octave, Key, Scale.*
- **Behavior:** These parameters represent discrete choices (e.g., selecting between 12 keys or various scales). The MIDI range (0-127) is divided into equal "zones."
 - *Example:* For the **Scale** selector, moving a knob will cycle through the available scales as the CC value increases.

D. Turntable Speed (CC Assignment)

The "SPEED" parameter allows for real-time manipulation of the turntable's rotational velocity, independent of the global BPM. This control is essential for achieving "time-stretching" effects or manual pitch-shifting on the fly.

- **CC Value Mapping:** The control is mapped linearly from **0.1x (minimum speed)** at CC value **0**, to **4.0x (maximum speed)** at CC value **127**. A value of approximately **30** corresponds to **1.0x (normal playback speed)**.
 - **Behavior:** When adjusted, the motor's current speed will smoothly ramp toward the target value, maintaining the organic "analog feel" of a physical platter.
 - **Recommended Controller:** For the best experience, we highly recommend using a **high-resolution physical fader** or a **large rotary encoder**. The precision of a fader allows for classic "pitch-riding" techniques similar to those used on professional DJ turntables.
-

Troubleshooting loopMidi

If your virtual ports (like loopMidi) are not appearing in the Standalone settings:

1. Ensure loopMidi is running **before** launching the application.
2. Check the **Options > Audio/MIDI Settings** menu within Skald to ensure the "Active MIDI Inputs" checkbox for your virtual port is ticked.

Read more about the dual-stack implementation below for technical details.

4. Standalone vs. Plugin Mode

- **Standalone:** Features a dedicated **Transport Bar** (Play/Stop and BPM Slider) at the top. Use the **REFRESH** button to select your physical MIDI interface or Virtual MIDI Cable (like loopMIDI or IAC). You can choose the midi outputs from the options (button at the top left) or do a "refresh" of the midi drivers of your system through the dedicated button at the bottom right and from here choose your output port to which the channels of your dots will be directed.
 - **Plugin (VST3/AU):** The transport is synced to your DAW. The BPM and Play/Stop functions are hidden as Skald follows the host's clock perfectly. MIDI is sent directly to the track's output or to other tracks via DAW routing.
-

5. DAW Setup Guide

To use Skald as a MIDI effect, follow these routing steps:

- **Ableton Live:** Place Skald on a MIDI track. Create a second MIDI track with an Instrument. In the second track's "MIDI From" menu, select the track containing Skald.
- **Logic Pro (AU):** Load Skald as a "MIDI FX" (found above the Instrument slot in the mixer).
- **Cubase/Nuendo:** Load Skald on a MIDI track. On your Instrument track, set the "MIDI Input" to "Skald - MIDI Out."
- **FL Studio:** Load Skald in the Wrapper. In the Wrapper settings (cog icon), set the "Output Port" to (e.g.) Port 1. On your target synthesizer, set the "Input Port" to Port 1.
- **Reaper:** Place Skald before an instrument in the FX chain, or use "Sends" to route MIDI to different tracks.
- **Studio One (PreSonus):** Load Skald on a MIDI/Instrument track. create a second Instrument track for your sound source. On the second track, set the **Instrument Input** to "Skald" (instead of "All Inputs" or your MIDI keyboard). Ensure the Record Arm or Monitor button is active on both tracks.
- **Samplitude / Sequoia (Magix):** Insert Skald as a VSTi on a track. On your target Instrument track, go to the **MIDI Input** settings in the Track Inspector. Choose **VSTi Out > Skald** as the source. Enable MIDI monitoring (the small speaker icon) to hear the generated notes.

(These DAWs are geared towards mastering/editing; the standard procedure via "VSTi Out" is how they handle internal MIDI routing.)

- **Pro Tools (Avid):** Create a **MIDI Track** and insert Skald. Create an **Instrument Track** with your desired plugin (e.g., Xpand!2). In the MIDI Track's **Output selector**, choose the name of your Instrument Track and the specific MIDI channel you wish to trigger. Note: Since Pro Tools handles MIDI FX differently, using Skald as a MIDI generator on its own track is the most stable method.

(Pro Tools doesn't like plugins that generate MIDI on the same track, so the two-track solution (MIDI + Instrument) is the one that guarantees the least problems for the user).

- **Reason (Reason Studios):** Create Skald in the Rack. Create your target Instrument (e.g., Europa or Thor). In the Rack, use the **CV/MIDI routing** on the back of the devices or use a "MIDI In Device" to route Skald's output to the instrument's sequencer input.

Make sure the destination tracks do not have "MIDI Thru" enabled to avoid infinite note loops if Skald is set to also receive external inputs (above all in Pro Tools and in Samplitude).

Best Practices: MIDI Channel Organization in Skald

We conclude this guide with a best practices framework for assigning MIDI channels. Following a standard, even if arbitrary, will help you stay on track when your projects become complex (with 20 or 30 dots on disk).

While you're free to do whatever you want, adopting a structure inspired by "General MIDI" or professional workflows will save you hours of troubleshooting.

MIDI Channel	Typical Destination	Recommended Dot type
Channel 1	Bass / Sub-Bass	Short gate, high velocity.
Channels 2-9	Lead / Keys / Arp	Variable gate, uses Swing a lot.
Channel 10	Drum Kit (Standard)	Variable velocity for "Ghosting".
Channels 11-15	Percussions / FX	Randomize and Low Probability.
Channel 16	Utility / Clock Out	Only Dots placed on the quarters.

Tips for a Clean Workflow

1. **Color and Channel:** while Skald assigns colors by ring, try to visually group the dots. For example, you might decide that the first three inner rings (the smallest ones) are always on Channel 10 for drums.
2. **The HUD** is your friend: When you use **ALT + Drag**, the HUD displays the channel number. If you're driving a multi-timbral sampler (like the MPC or Kontakt), make sure the number in the HUD matches the instrument's "MIDI Slot."
3. **Channel Gate Management:**
 - On the **Synth** channels (**Lead/Pad**), experiment with **CTRL + Drag** to create very long arcs. This will create interesting legato lines as the record spins.
 - On the **Drum channels (Ch 10)**, keep the Gate at a minimum (almost invisible arc) to avoid sample overlap that could "bog down" the sound.
4. **Export:** If you choose to drag the generated MIDI into your DAW, remember that the resulting clip will be either "Type 0" or "Type 1." DAWs will typically automatically separate notes onto different tracks based on channel if you import them correctly.

6. Technical Insights & Logic

Skald is built on the **JUCE Framework** using a high-precision sample-accurate timing engine.

- **The Rotation Engine:** instead of a standard "grid," Skald calculates triggers based on the floating-point angle of the dots relative to the current playhead position. This allows for the **Swing** and **Reverse** functions to work seamlessly without losing sync.
- **The Hyper-Euclidean Algorithm:** uses a modified Bjorklund algorithm that doesn't just calculate linear pulses, but maps them onto a polar coordinate system, allowing for "spiral" patterns when the **Depth** parameter is increased.
- **UI Rendering:** uses Hardware Acceleration (OpenGL/CoreGraphics) to ensure the 60fps rotation of the turntable does not strain the CPU, leaving more power for your synthesizers.
- **Export Logic:** the MIDI Export function translates the polar coordinates (angle and ring index) of every dot into a linear MIDI sequence. It calculates the exact onset and duration based on the current BPM and Speed settings, ensuring that the exported file perfectly matches the "groove" you heard during the live generation.
- **MIDI LEARN LOGIC:**

Continuous Controls(BPM, Velocity, Shift)	Linear mapping 0-127
Loop Switches (Div, Oct)	Trigger or zone division
List Switches (Key, Scale, Depth)	Division of the MIDI range into "slots" (zones)
Switches (Rev, Motor, Play)	Toggle logic (Threshold 64)

- **The Turntable:** the turntable in MIDI Learn is absolutely essential for DJing. To make it truly "playable" via MIDI like a real vinyl record, paramID == "motor" and paramID == "rev" were first included, then scratch and direct rotation management was added, as well as a specific scratch case within the applyMidiControl logic. If the user moves a MIDI jog wheel (CC), Skald will move the record back and forth proportionally. This turns a MIDI controller into a real scratch platter.

- **SAVE:** MIDI Learn assignments are saved within the existing `getStateInformation` and `setStateInformation` functions, so when you press "SAVE" in the Skald interface, the Processor writes a `.ttp` file that saves in this order:
 1. Dot Position (Angle, Ring, Color, MIDI Channel, Velocity, Gate).
 2. Slider State (BPM, Probability, Swing, etc.).
 3. Algorithmic Ring State (Pulses, Steps, Depth, Shift).
 4. The entire MIDI Learn Assignment table.

This way, when you load a `.ttp` file, Skald not only repositions the notes on disk, but also instantly configures your hardware controller for that specific preset.

- **Version Name & IDs:** Since version 1.2.1 the name of the app (standalone and plugin) has been changed from "Skald" to "Skald-Reworked" and the identifiers codes have also been changed for both the Bundle ID and the AAX and LV2 plugins, so when loading version 1.2.1 or later the DAW may not recognize the plugin as the same as previous versions but think of them as different plugins.

- **RtMidi Integration High-Performance MIDI Engine (RtMidi)**

"Under the Hood": to ensure the lowest possible latency and maximum stability during live performance, Skald has transitioned its core MIDI handling to the RtMidi library. This architectural change bypasses standard plugin-wrapper bottlenecks, offering several key advantages:

Direct Hardware Communication: by utilizing a dedicated MIDI thread, the plugin establishes a direct link with your hardware controllers. This ensures that heavy DAW processing or high CPU loads do not "choke" your MIDI input.

Thread-Safe Atomic Updates: parameters like *Turntable Speed* and *Trigger Probability* are now managed via *Atomic Variables*. This allows the UI, the MIDI callback, and the Audio Engine to communicate simultaneously without the risk of crashes or "glitchy" audio artifacts.

Sample-Accurate Response: even though the MIDI is processed in a separate thread, the engine calculates the exact sample-offset for every message, ensuring that your manual scratches and probability shifts are perfectly aligned with the audio buffer.

Hardware Agnostic: RtMidi provides a robust, cross-platform layer that ensures consistent behavior across different Operating Systems and MIDI interfaces, from vintage 5-pin DIN setups to modern USB-MIDI controllers.

Technical Overview: Dual-Stack MIDI Implementation

(Current State version 1.2.2)

1. Architecture Approach

The system currently employs a **Hybrid MIDI Stack** designed to maximize compatibility across the evolving Windows MIDI landscape. It simultaneously initializes two distinct API backends through the **RtMidi** library:

- **WinRT (UWP) Backend:** Targets the modern *Windows MIDI Services*. It provides multi-client capabilities and support for MIDI 2.0 and Bluetooth LE MIDI.
- **WinMM (Legacy) Backend:** Targets the older *Windows MultiMedia* API. This is included specifically to capture "invisible" devices to WinRT, such as virtual drivers (e.g. **loopMIDI**) and older USB-MIDI hardware without UWP-certified drivers.

2. Threading and Callback Management

Since each **RtMidi** instance (UWP and Legacy) spawns its own high-priority thread, the application implements a **Dual-Threaded Input Model**:

- **Collision Avoidance:** A `juce::ScopedLock` is utilized within the `midiCallback` to prevent race conditions when both stacks attempt to access the `midiMappings` map or modify processor parameters (like `speed` or `probability`) simultaneously.
- **Atomic Synchronization:** Critical real-time parameters are stored in `std::atomic` variables (`midiSpeed`, `midiProbability`) to ensure that even if a message is processed on a background MIDI thread, the audio `processBlock` reads the latest value without locking the audio thread.

3. Initialization and COM Requirements

To support the WinRT stack, the application enforces a **Multi-Threaded Apartment (MTA)** model.

- The call to `CoInitializeEx(nullptr, COINIT_MULTITHREADED)` in the `SkaldProcessor` constructor ensures that the COM environment is ready to handle the asynchronous nature of WinRT MIDI device enumeration and notification, which is notoriously slower and more complex than the legacy WinMM system.

4. Current Limitations (The "Shadowing" Issue)

Current observations indicate that when **Windows MIDI Services (WMS)** is active and configured with Loopback Endpoints, the OS-level abstraction layer may prioritize or "shadow" legacy WinMM devices.

This results in the **WinRT** stack successfully reporting modern **WMS** ports while the **WinMM** stack fails to retrieve handles for legacy virtual devices like **loopMIDI**, effectively nullifying the benefits of the dual-stack approach in certain system configurations.

